ImageClassification

Dimitrios Pakakis

March 2026

1 Introduction

In this project, i conduct a comprehensive analysis of the behavior of the Knn and nearest centroid classifier for image classification. Using the FashionMNIST dataset, experiments that explore the impact of different hyperparameters as well as data preprocessing techniques, like the way to convert images into vector representations, are implemented.

The primary objective is to better understand how these algorithms perform under different configurations and conditions. By evaluating their behavior, i aim to identify ways to optimize their classification performance while also improving the execution time required for the task.

2 Dataset

Fashion-MNIST is a dataset of Zalando's article images, consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from 10 classes. Each image is 28 pixels in height and 28 pixels in width, for a total of 784 pixels in total. Each pixel has a single pixel-value associated with it, indicating the lightness or darkness of that pixel, with higher numbers meaning darker. This pixel-value is an integer between 0 and 255.

3 Evaluation Metrics

For the evaluation i will use:

- Accuracy: Number of correct predictions
- Macro Precision: average for all the classes of how often the model predictions are false (FP)
- Confusion Matrix: It compares the predictions made by the model with the actual results and shows where the model was right or wrong.

For the accuracy and precision 100% means perfect performance.

4 Normalization

Normalization of the pixels is neccesery. Without normalization we will face Datatype Overflow. The values of pixels are between 0-255 and they are stored as unit8 (Unsigned Integer 8-bit). In the euclidean distance, when we calculate $a^2 + b^2 - 2ab$ then a^2 and b^2 may take values > 255 (e.g $255^2 = 65,025$). These numbers are bigger than the numbers unit8 can store, so the matrix distance is full of wrong numbers. Using Normalization we solve the datatype overflow problem.

Additionally, the time complexity is the time needed for the prediction. Without Normalization the delay is big, so the response time, and this makes the algorithm unefficient. Therefore, normalization is neccesery for the latency reduction.

We want to keep the values between 0 and 1, so the min-max scaling is the best way. Additionaly the Z-score is efficient and can be used. I will use the min-max scaling (pixels/255.0) through all the analysis.

5 Standard Knn Classifier Analysis

5.1 Standard K-nn Time complexity

I will use Knn with k=11, distance metric = manhattan and as a feature extraction i will use flattening. By flattening the images i convert 28x28 to 784 vector. The feature

extraction is main factor for the time needed. Lets say that this is a simple implementation of the algorithm and i want to see the prediction time needed for 10,000 samples so i will not focus on algorithm's efficiency.

For 10,000 samples the time needed is: 361.179 sec and for 1 sample 0.036 sec.

At first glance, 361.179 sec is not a big problem. However 28x28 images are small and not real-world examples. While the image dimension gets bigger the time complexity increases. Using 0.036 sec/image as a reference point, i will try to reduce the time complexity and compare with the original.

5.2 K-nn using k=1

K-nn algorithm with k=1 is a way to see what overfiting means. Every sample creates a region, where if a sample is in this region, then this is the nearest neighbor. As a result the decision boundary is complex. The knn algorithm does not have a training phase, so we can't compare training-validation loss. However, we know that with k=1 the training loss is 0, because every point is the first neighbor of itself. Using this model, with flattening the images as feature extraction technique, to new unseen data (test set) the performance was reduced. Specifically from accuracy 100% (train set, seen data) the Accuracy dropped to 84.9%, while macro Preccision score is 85.2%. This performance reduce indicates overfiting and big variance. The training set is the 100% factor of the decision boundaries, small changes on the training set have big impacts on the decision boundaries, this is where we can detect a big variance.

In order to vizualize the complex decision boundaries, i calculate the top-bottom mean brightness of the image so we can have 2D features. I selected randomly 5 indices from 4 classes, and i plot the decision boundaries for each class, using k=1 and k=7.

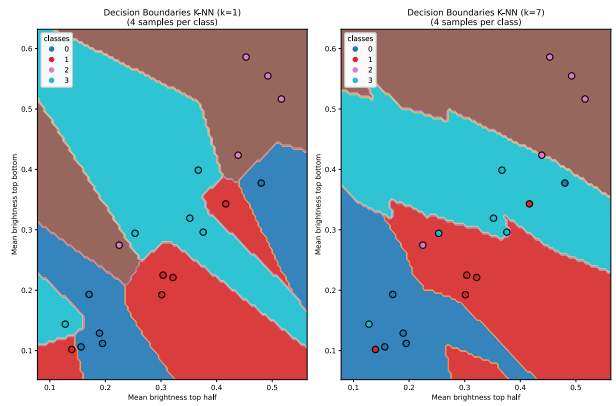

Figure 1: In the figure we can detect that the k=1 diagram creates many more boundaries, than the classes, around the samples. Conversely, the k=7 algorithm creates simpler boundaries and in a number close to the number of classes

6 1st try Folds Cross Validation for K-nn

The cross-validation is important so we can see the algorithms performance in different validation sets and calculate the average performance for all the folds. These are better metrics than using a test set that could be biased. I will conduct a 10 cross-validation for the knn-algorithm using half the data(30,000 samples). This technique is time-consuming and it is preferred to impelemnt it on a smaller subset of the data. It is neccesery to keep the class balance in the dataset.

6.1 Feature Extraction

As a technique to convert the 28x28 images to a vector i will use flattening. Flattening is a technique where we keep all the information of the image, in contrast to top-bottom mean brightness where 2 numbers try to describe the whole image. However the flattening has 2 main problems and in section 6.3 we will see that sets a top boundary on the algorithms performance. The 1st problem is that flattening hides the spatial information of the image. Since the pixels are placed in a single sequence, the spatial relationship between them is no longer explicitly preserved. As a result, it becomes unclear which parts of the image are close to

each other.

For example, whether a sleeve is short or long becomes encoded somewhere inside this large vector representation rather than being directly represented through spatial structure. Consequently, this information is harder for the algorithm to interpret. Indeed, as we will observe later, this leads to repeated misclassifications in certain classes.

Another important problem is that as the dimensionality increases, the concept of distance becomes less meaningful. In high-dimensional spaces, the distance to the nearest neighbor and the distance to the farthest neighbor tend to converge toward similar values.

As a result, the distinction between close and distant points becomes less clear. This phenomenon significantly affects distance-based algorithms and limits the performance of the algorithm, since it becomes harder to reliably identify truly similar samples.

6.2 Hyperparameters

The hyperparameters i want to test are k : [5,7,9,11] and distance metric : [euclidean, cosine distances, manhatan distance]. Thus i will run k*distance = 12 times the 10 cross validation. A very time consuming proccess but bigger impact for the analysis.

6.3 Results

For every combination of the parameters i calculate the metrics (mean for every fold) and then i can select the optimal. With the folds technique we can find the best hyperparameters combination. Using the Knn algorithm with the optimal parameters i will plot the confusion matrix and False-True predictions on the fashionMNIST test set.

Results:

- K=5 , distance = euclidean

  Accuracy: 84.3%, Macro Precision: 84.6%

- K=5 , distance = cosine

  Accuracy: 84.7%, Macro Precision: 85.3%

- K=5 , distance = manhatan

  Accuracy: 84.9%, Macro Precision: 85%

- K=7 , distance = euclidean

  Accuracy: 84.1%, Macro Precision: 84.5%

- K=7 , distance = cosine

  Accuracy: 84.4%, Macro Precision: 85%

- K=7 , distance = manhatan

  Accuracy: 84.8%, Macro Precision: 84.9%

- K=9 , distance = euclidean

  Accuracy: 84%, Macro Precision: 84.4%

- K=9 , distance = cosine

  Accuracy: 84.1%, Macro Precision: 84.8%

- K=9 , distance = manhatan

  Accuracy: 84.6%, Macro Precision: 84.8%

- K=11 , distance = euclidean

  Accuracy: 83.8%, Macro Precision: 84.2%

- K=11 , distance = cosine

  Accuracy: 83.9%, Macro Precision: 84.7%

- K=11 , distance = manhatan

  Accuracy: 84.4%, Macro Precision: 84.5%

From the results we cann see that the combinations: k=5, distance=manhatan and k=7, distance=euclidean are the best. To find the best between them we will try the algorithm to the full dataset (60,000 training, 10,000 test). The combination k=5 and distance=manhatan is the optimal combination with accuracy=86.2% and macro precision=86.3% while the combination k=7 and distance=euclidean achieved accuracy=85.3% and macro precision=85.7%.

6.4 Plots

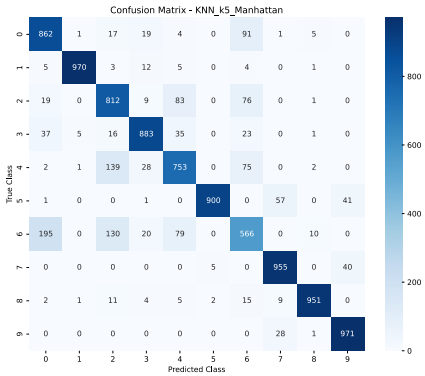

Figure 2: Confusion Matrix

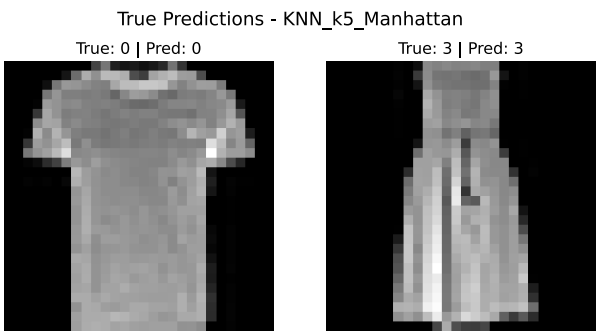

Figure 3: True Predictions

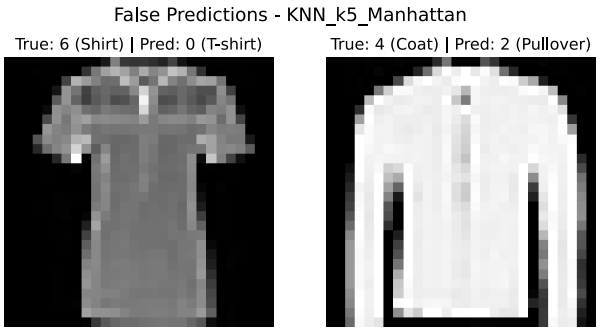

Figure 4: False Predictions

6.5 Comments

We can detect that the performance does not have significant changes as we vary the hyperparameters. This behaviour is predictable. The knn algorithm using flattening has a maximum performance that we can reach. There are two main reasons for this behaviour. The reasons are the reasons i described in section 6.1. The reduction in the significance of distance can be observed through the hyperparameter k. The choice of 5 neighbors proved to be the best option. As the number of neighbors increased, performance decreased because the distances between points were very similar.

As a result, it became more likely that some neighbors belonged to the wrong class, even though they appeared close in the 784-dimensional space. This effect makes the nearest-neighbor selection less reliable and negatively affects the classification performance.

Another important issue is the absence of spatial information. From both the confusion matrix and the examined examples, we observe that the algorithm makes repeated errors in specific classes.

In particular, it frequently confuses class 6 (shirt) with class 0 (T-shirt). This is understandable, since these two types of clothing are visually very similar. When spatial information is not preserved and only the pixel intensity information remains, it becomes much more likely for the model to make incorrect predictions.

The same pattern can also be observed between classes 4 (Coat) and 2 (Pullover), where the visual similarity between the items leads to repeated misclassifications.

Through this analysis, we can conclude that the main issue is no longer the algorithm itself, but the data, and more specifically the way the images are represented.

Therefore, the focus shifts from improving the algorithm to improving the data representation. In the following steps, we will concentrate on how we can improve the model's performance by modifying the way the images are represented.

7 2nd Try Folds Cross Validation for K-nn

In this cross validation we will change only the feature extraction. We want to analyze how the feature extraction

derives the whole procedure and if we can overcome the 85% limit.

7.1 Feature Extraction

This time, instead of simply placing the pixels in a sequence, we will create a representation that preserves meaningful structure. This representation will be constructed using the PCA algorithm.

The PCA algorithm seeks to identify the directions of greatest variance in the data. It does this by computing the eigenvectors, which define the directions, and the eigenvalues, which indicate the importance of each direction from the convariance matrix.

Next, the algorithm selects the components with the largest eigenvalues, as these capture the most significant variations in the dataset. This way, PCA reduces dimensionality while preserving the most informative structure of the data.

With PCA, we are able to significantly reduce the dimensionality of the data. As the number of dimensions decreases, we gain improvements in computational time. More importantly, the concept of distance becomes more meaningful.

This helps recover useful structural information, since images with similar shapes tend to have smaller distances in the transformed space. As a result, the algorithm can better distinguish between different classes.

In addition, the PCA algorithm helps reduce image noise. Pixels that belong to the background will no longer strongly influence the training set, because PCA treats them as components with very low variance, and therefore they contribute little to the final representation of the data.

However, this selection introduces another problem: the hyperparameter issue. PCA retains the n principal components, where n must be defined by the user.

Choosing the right n is crucial, as it determines how much information is preserved versus how much dimensionality is reduced. This creates another space for analysis, where we need to experiment to find the optimal number of components that balances performance and efficiency.

However, the PCA algorithm is not a black box. We know that it produces ordered axes based on their importance. The first principal component may explain, for example, 20% of the dataset's variance, the second principal component 10%, and so on.

This means we can tell the algorithm how much of the dataset's variance we want to retain. For example, we might decide to retain 95% of the variance, and PCA will automatically calculate how many components are needed to achieve this. The higher the percentage of information (variance) we want to retain, the more components the PCA algorithm will return.

One approach is to apply PCA to our data and initially set it to produce 784 components. Then, we can use the function: `pca.explained_variance_ratio_`, which returns the amount of information (variance) explained by each component, starting from the most important one. Therefore, we can create a plot where the x-axis represents the components and the y-axis represents the amount of information gained. This allows us to clearly see how much each additional component contributes and helps us choose an appropriate number of components.

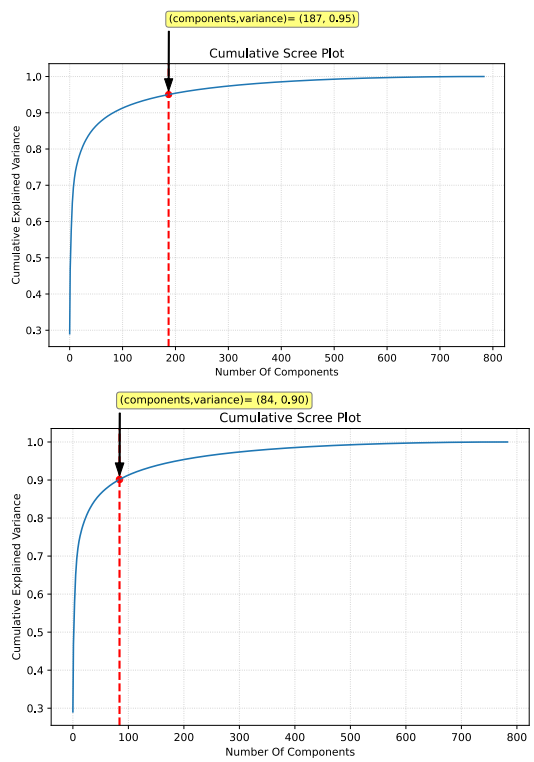

Figure 5: To retain 95%, first image, we need 187 components. To retain 90%, second image, we need 84 components

We observe that after 90% variance, the number of components increases very rapidly, while the gain in variance is minimal. For example, to retain 90%, we need 84 components, whereas for 95%, we need 184 components. This raises an important question: is it worth more than doubling the dimensionality for just an additional 5% of variance? The only way to answer this is through experimentation.

However, in the cross validation already perform 12 iterations, adding these two additional values would increase the total to 24 iterations. Unfortunately, if we want a proper analysis, this step is necessary. We can't avoid trying all the combinations because changing the retained variance directly affects the dimensionality, which as we said it has a significant impact on the problem.

So we need to manage this computational cost. First, we perform 24 times cross-validation runs on a smaller subset of the dataset (10,000 samples). From these, we select the 2 best combinations we found using 90% information retrieval and 95%, based on accuracy. Then we will have 4 combinations, where two of them are with 90% and the other two 95%. Then, we apply cross-validation again using these top configurations on the whole dataset.Finally, we calculate the accuracy/precision/confusion matrix for these 4 combinations. If the best-performing configuration has 95% on PCA , we will examine if the computational cost is worth the improvement. For example we doubled the dimensions, so the time needed is much more. If the combination with the 95% has 1-5% more accuracy than the 90% we can say that it is not worth it. However in this dataset we can sustain the cumpotational cost, but in real life problems this is not an option. The best combination will be evaluated on the final test set.

To implement the cross validation using the pca we must take care of 2 things. First, we must change the folds loop, because we dont have to use PCA for the same fold and same hyperparameters because pca calculate the same thing. The loop must start with the folds , use PCA and then use the k and distance hyperparameters. This procedure can be shown in the notebbok and it is more clear what i mean. Second, we must be very carefull not to train the PCA to the dataset before splitting, because we will have data leakage. We fit only to the train set and we transform both train and test set.

7.1.1 Results

- variance=95% , k=11, distance = manhattan,

  Accuracy: 82.69%

- variance=95% , k=7, distance = manhattan,

  Accuracy: 82.68%

- variance=90% , k=11, distance = cosine distance,

  Accuracy: 81.34%

- variance=90% , k=11, distance = manhattan,

  Accuracy: 81.22%

7.2 Hyperparameters

For the new cross validations we will use the above combinations.

7.3 Results

The results on the 10 cross validation using the whole training dataset. The best combination will be evaluated at the test set.

- variance=95% , k=11, distance = manhattan,

  Accuracy: 86.32%, Macro Precision = 86.55%

- variance=95% , k=7, distance = manhattan,

  Accuracy: 86.53%, Macro Precision =86.74%

- variance=90% , k=11, distance = cosine distance,

  Accuracy: 86.54%, Macro Precision: 86.54%

- variance=90% , k=11, distance = manhattan,

  Accuracy: 86.57%, Macro Precision = 86.66%

  We can see that the combination PCA: 90%, k=11 and distance = manhatan has the best accuracy. The combination PCA: 95%, k=7 and distance = manhatan has 0.08 better macro precision but this is surely not a good reason to pay the time complexity for this. So we found the best combination. We will evaluate at the test set and plot confusion matrix and True/False preidctions.

7.4 Knn using k=11, distance = manhattan and PCA with 90% variance retrieval

Our algorithm achieved Accuracy = 86.3% and Macro Precision = 86.35%. Using PCA we managed to overcome the limit of 85%. Our algorithm needed 49.433 sec for 10,000 samples, so 0.005 sec for 1 sample. We can see that the time complexity has improved from the starting algorithm at section 5.1 and the accuracy has improved also.

7.5 Comments

Our algorithm achieved Accuracy = 86.3% and Macro Precision = 86.35%. Using PCA we managed to overcome the limit of 85%. Our algorithm needed 49.433 sec for 10,000 samples, so 0.005 sec for 1 sample. We can see that the time complexity has improved from the starting algorithm at section 5.1 and the accuracy has improved also. However, by examining the confusion matrix and the precision per class,which is very low for classes 2,6, and the false predictions, we can observe that the same pattern of errors between specific classes still persists. Looking at the matrix more closely, it becomes clear that the main factor reducing accuracy is this confusion between similar classes, while the performance on the remaining classes is quite strong.

Therefore, although PCA significantly improved both computational efficiency and overall performance, it does not seem to resolve the core issue of the problem. This suggests that it may not be the most suitable way to represent our data.

If we consider the nature of the images, we can understand that the problem mainly arises with visually similar clothing items, which differ only in subtle features such as sleeve length or height.

One possible idea, then, is to compute the average intensity of each row and each column. This type of representation could help capture these structural differences and potentially improve the model's ability to distinguish between such similar classes.

7.6 PLots

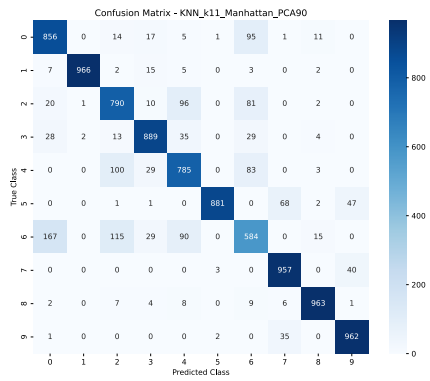

Figure 6: Confusion Matrix

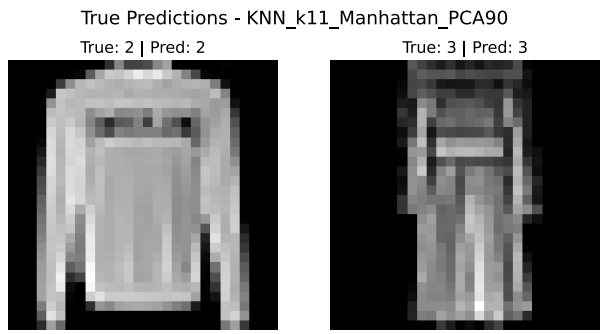

Figure 7: True Predictions

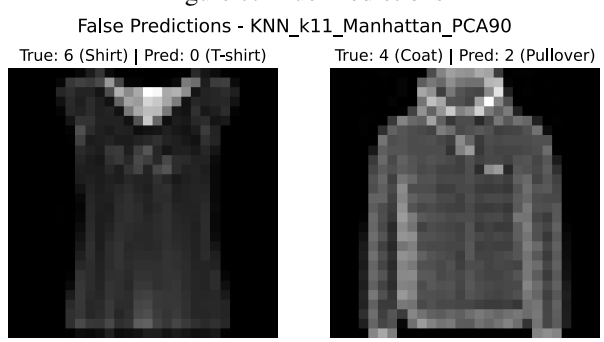

Figure 8: False Predictions

8 3rd try Folds Cross Validation for K-nn

We will keep the same combinations, but this time the feature extraction will be implemented using the mean

brightness per axis.

8.1 Feature Extraction

With flattening and PCA we detected a pattern of wrong predictions. A possible solution to overcome this problem is calculating the mean brightness per rows, so the clothes with sleeves will have a bigger brightness mean in the rows with sleeves, and calculating the mean per column, so we can distinguish short clothes from long clothes.

8.2 Hyperparameters

The hyperparameters i want to test are k : [5,7,9,11] and distance metric : [euclidean, cosine distances, manhatan distance]. Thus i will run k*distance = 12 times the 10 cross validation.

8.3 Results

For every combination of the parameters i calculate the metrics (mean for every fold) and then i can select the optimal. With the folds technique we can find the best hyperparameters combination. Using the Knn algorithm with the optimal parameters i will plot the confusion matrix and False-True predictions on the fashionMNIST test set.

Results:

- K=5 , distance = euclidean

  Accuracy: 80.64%, Macro Precision: 80.81%

- K=5 , distance = cosine

  Accuracy: 81.76%, Macro Precision: 82.16%

- K=5 , distance = manhatan

  Accuracy: 81.94%, Macro Precision: 82.06%

- K=7 , distance = euclidean

  Accuracy: 80.84%, Macro Precision: 81.03%

- K=7 , distance = cosine

  Accuracy: 81.66%, Macro Precision: 82.16%

- K=7 , distance = manhatan

  Accuracy: 82.03%, Macro Precision: 82.13%

- K=9 , distance = euclidean

  Accuracy: 80.87%, Macro Precision: 81.05%

- K=9 , distance = cosine

  Accuracy: 81.41%, Macro Precision: 81.98%

- K=9 , distance = manhatan

  Accuracy: 81.98%, Macro Precision: 82.07%

- K=11 , distance = euclidean

  Accuracy: 80.85%, Macro Precision: 81.04%

- K=11 , distance = cosine

  Accuracy: 81.33%, Macro Precision: 81.92%

- K=11 , distance = manhatan

  Accuracy: 81.98%, Macro Precision: 82.06%

From the results we cann see that the combination: k=7, distance=manhatan is the optimal. We will try the algorithm to the full dataset (60,000 training, 10,000 test). The combination k=7 and distance=manhatan achieved accuracy=82.39% and macro precision=82.5%

8.4 Plots

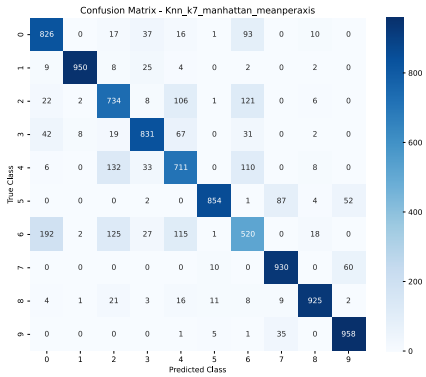

Figure 9: Confusion Matrix

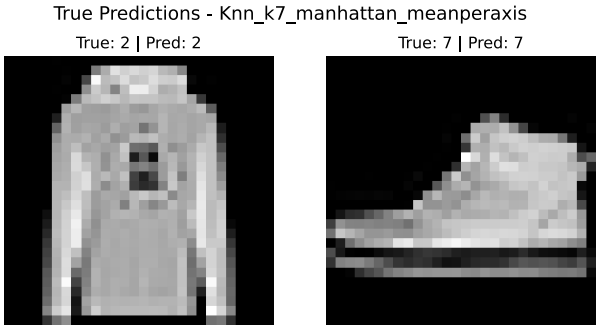

Figure 10: True Predictions

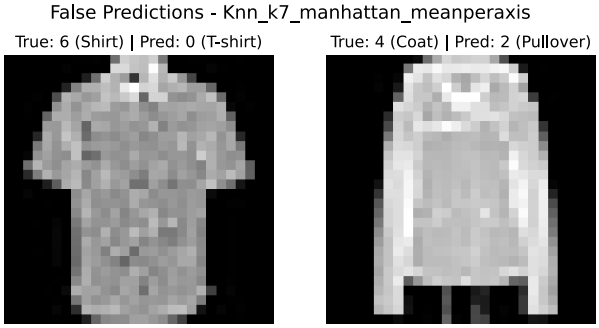

Figure 11: False Predictions

8.5 Comments

The accuracy we achieved is lower than when using PCA for feature extraction. Also, the problem with the specific misclassifications was not solved; in fact, it seems to be even worse. So, we can say that we will not keep this approach. In general, KNN relies only on distances, which makes it hard to capture small details in the images. Because of this, its performance reaches a limit. We could try other ways to represent the data, but it is unlikely that we would see a big improvement. Therefore, from our analysis, the best combination is: k = 11, Manhattan distance, PCA with 90% variance This setup gives the best balance between accuracy and speed.

9 Try our algorithm in another dataset

We have pushed our algorithm to a certain point, and we can assume that if we apply it to a similar problem, it will achieve reasonably good performance (Generalization). It may not be the best possible result, but it should still provide a solid outcome. To further evaluate it, we will use the MNIST dataset, specifically its test set, in order to examine how the algorithm performs on slightly different data. It is obvious that, another dataset means another techniques, feature extractions etc. However when training a model it is a good technique to check the performance on a similar task, soo one can be sure that the model is not overfitted. Knn has not a training phase and coefficients. we do this mainly to demonstrate how image classification would be properly handled using a different algorithm.

9.1 Results

9.1.1 Metrics

Our algorithm achieved an accuraacy of 96.42% and Macro Precision of 96.50%.

10 Plots

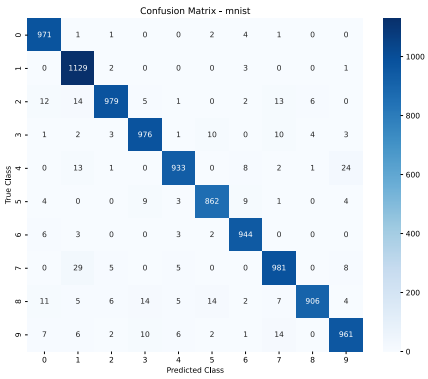

Figure 12: Confusion Matrix

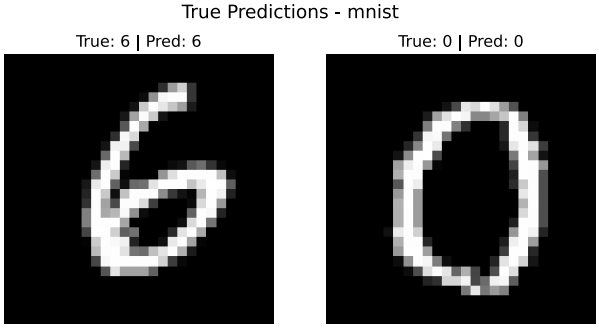

Figure 13: True Predictions

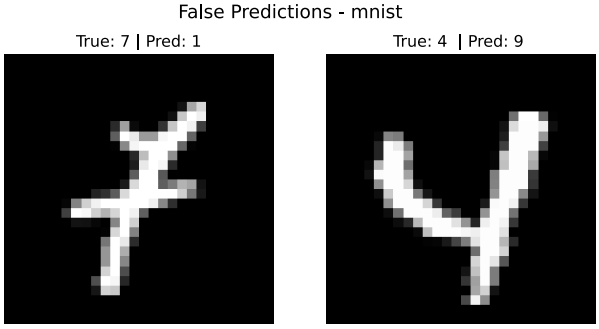

Figure 14: False Predictions

10.1 Comments

The performance of our algorithm appears to be much better on the MNIST dataset. The accuracy is significantly higher compared to what we achieved on our own dataset.

This makes the core problem clearer. Certain classes, in other words certain clothing categories are very similar, which leads to many misclassifications. In contrast, in the MNIST dataset, although there is some similarity between digits, the algorithm is still able to distinguish them more effectively.

11 Nearest Centroid Classifier

Although the KNN algorithm uses labels, we cannot fully consider it a truly supervised algorithm, since it only uses them indirectly through the k nearest neighbors. As a result, we do not fully take advantage of the available label information to help the model understand that visually similar items may belong to different classes.

To improve performance, we will use the nearest centroid classifier. This algorithm creates a number of centroids equal to the number of classes, where each centroid represents a general description of its class.

When a new sample is introduced, the algorithm computes its distance from each centroid, and the smallest distance determines the predicted class. Unlike KNN, this method makes more direct use of the training labels, since each centroid is formed from samples of the same class.

These centroids may help the model better distinguish between very similar clothing items, by effectively increasing the distance between classes.

12 1st try

We will train the algorithm to the train set and evaluate on the test set. We will use flattening as feature extraction technique and euclidean distance.

12.1 Results

The Nearest Centroid Algorithm achieved Accuracy: 67.68% and Macro precision: 68.09%. The training time for the train samples was 0.441 sec and the prediction time for the test set was 0.015 sec.

12.2 Plots

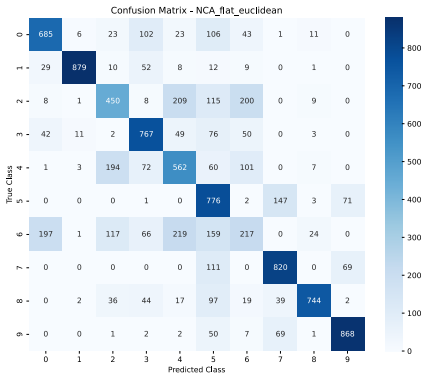

Figure 15: Confusion Matrix

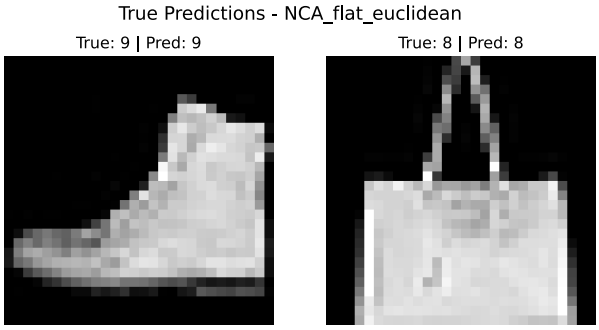

Figure 16: True Predictions

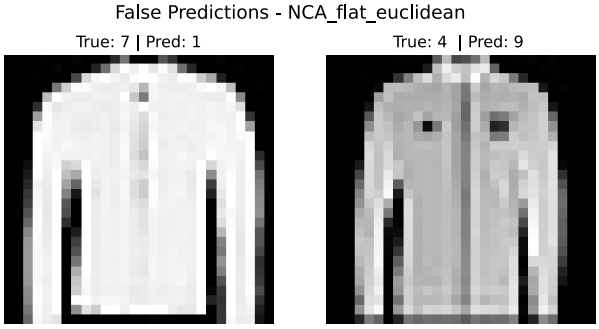

Figure 17: False Predictions

12.3 Comments

It seems that the performance of the algorithm was not what we expected. Even though, in theory, we thought we had addressed the problem, in practice KNN performed much better.

From the confusion matrix, we observe that the misclassifications not only increased but also spread across more classes, whereas with KNN, the false predictions were mostly concentrated in specific classes.

We will also try applying PCA for feature extraction, although it is unlikely that this is the root cause of the problem. In the comments section of the second attempt, we will further analyze what the actual issue is.

Despite this, the computational efficiency is noteworthy. The algorithm is much faster than KNN, especially in terms of prediction time, which has been significantly reduced compared to the optimized version of KNN. Compared to the original KNN implementation, the improvement in speed is huge.

13 2nd try

We will train the algorithm to the train set and evaluate on the test set. We will use PCA with 90% variance retrieval as feature extraction technique and euclidean distance.

13.1 Results

The Nearest Centroid Algorithm using PCA achieved Accuracy: 67.59% and Macro precision: 68.%. The prediction time for the test set was 0.015 sec.

13.2 Plots

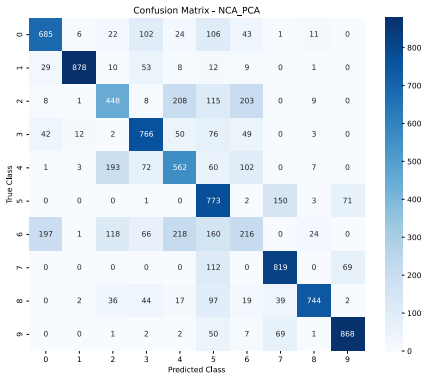

Figure 18: Confusion Matrix

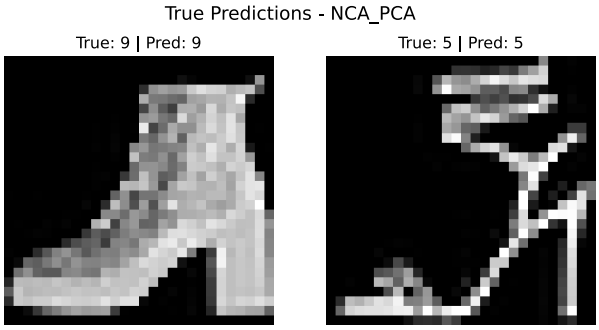

Figure 19: True Predictions

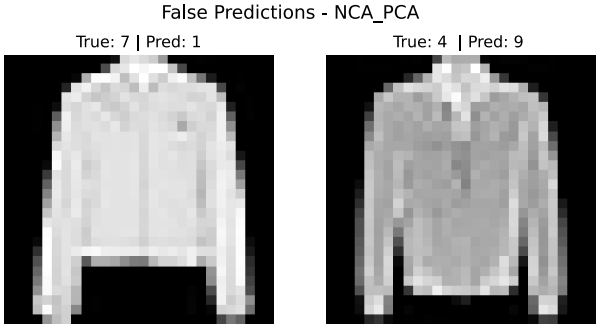

Figure 20: False Predictions

13.3 Comments

Indeed, we were not able to improve anything. The performance remains exactly the same, and the algorithm does not seem to benefit from the use of PCA.

A possible explanation for this behavior is that the nearest centroid classifier creates one centroid per class. However, in reality, each class may contain subclasses—that is, variations within the same category. For example, a "shoe" class may include both sneakers and sandals, which can be quite different from each other.

As a result, the centroid created for a class may end up being a poor representation, since it is influenced by diverse types of items and may not closely resemble any actual sample. Therefore, the Nearest Centroid CLassifier has a poor performance in contrast to Knn. The accuracy is low not only in contrast to Knn, but in general 65% accuracy is not a good metric. However the time complexity is very good, but in my opinion i would pay the cost of time to improve this poor performance.

To investigate and potentially address this issue, we will use K-Means clustering within each class. Specifically, for each class, we will create three subclasses. This way, instead of having 10 centroids, we will have 30 centroids, which may allow the model to better capture the internal structure of the data and improve performance.

14 Kmeans and Knn

By using K-Means, we will create 30 centroids, meaning that each class will have 3 centroids. As a result, we construct a new dataset of 30 samples, where each sample represents the center of a subclass. We will then apply KNN with k = 1 to determine the class of a new sample. Although we said that k = 1 is typically the definition of overfitting, in this specific case it turns out to be the best choice. Since each class has only 3 samples (centroids), we are essentially limited to choosing k = 1 or k = 3. In our case, it is sufficient for a sample to be close to at least one centroid of its class, so k = 1 is the most appropriate option. This approach may help us overcome the issue of similarity both between classes and within the same class. For feature extraction, we will use PCA before applying K-Means, in order to reduce dimensionality and make distances more meaningful.

Of course, this idea requires further analysis, but we will stop at this point. There are several parameters we could tune—such as the number of clusters in K-Means, the value of k in KNN, and the distance metric—but for now we choose: clusters = 30, k = 1, distance metric = manhattan

14.1 Plots

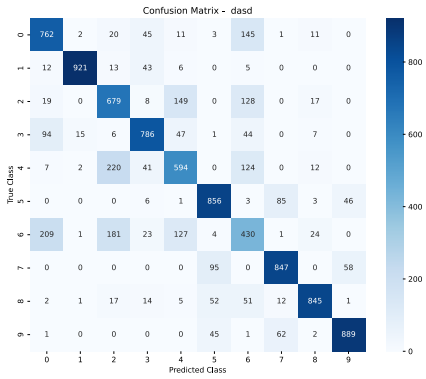

Figure 21: Confusion Matrix

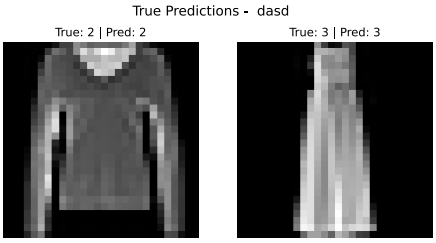

Figure 22: True Predictions

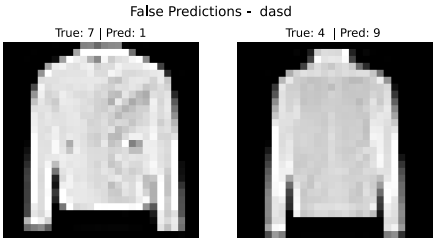

Figure 23: False Predictions

14.2 Results

The PCA->KMeans->Knn pipeline achieved Accuracy: 76.09% and Macro precision: 76.34%.

14.3 Comments

We managed to improve performance compared to the nearest centroid classifier, but we were not able to surpass the performance of the KNN model we developed earlier.

We reduced the misclassifications that were previously spread across all classes, but they still remain to some extent. Additionally, class 6 continues to produce frequent errors, a pattern that we have consistently observed throughout this analysis.

15 Conclusion

Image classification is a naturaly complex problem, but when the data does not contain clear distinctions, it can impose a performance ceiling on the algorithm.

In this case, we can largely "blame" the accuracy limitat to the data itself, as we observed that performance on the MNIST dataset was very good—even without extensive hyperparameter tuning or specialized feature extraction. This highlights how important the quality and separability of the data are.

At the same time, distance-based algorithms struggle to capture small details, which is why more advanced methods such as Convolutional Neural Networks (CNNs) and U-Net architectures are commonly used in real-world applications.

Throughout the analysis, we consistently observed that class 6 showed strong similarities with other classes, leading to repeated misclassifications. Similar behavior was also present in other classes.

In conclusion, we achieved an accuracy of 86.3% using: PCA with 90% variance retention, k = 11 neighbors, Manhattan distance.

This represents the best result obtained under the given conditions.